Jornada 4: SoC 2-Core con Mailbox

Daniel Rosso - Pablo Madoery

Fundación Fulgor

Miércoles 9 de setiembre de 2026

Contenidos

1. PRBS y BER
2. Motivación
3. El Mailbox
4. BRAM Dual-Port
5. Estructura RTL
6. Mapa de Memoria
7. Firmware
8. Control de la tasa de error
9. Ordenamiento
10. Testbench
11. Laboratorio

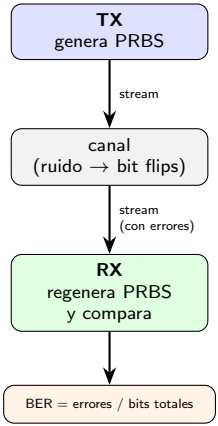
¿Qué es un BERT? PRBS y BER

Bit Error Rate (BER):

- En cualquier enlace real (cable, radio, óptica) el ruido puede invertir bits en tránsito
- BER = bits recibidos mal / bits transmitidos, expresado en ppm o como 10^{-n}
- Es la métrica estándar para medir la calidad de un enlace de comunicaciones

El problema de medirlo:

- Para contar errores, el receptor necesita saber *qué bit debía llegar* sin que el transmisor se lo diga aparte
- Un **BERT** (*Bit Error Rate Tester*) resuelve esto transmitiendo una secuencia **conocida de antemano** por ambos lados



PRBS: Pseudo-Random Binary Sequence

Una secuencia **determinística** (se puede regenerar exactamente igual dos veces) que sirve como estímulo realista para probar un enlace.

El RX **no recibe la secuencia aparte** para comparar: la regenera de forma independiente, bit a bit.

BER en ppm: ¿qué significa exactamente?

Definición

$$\text{ppm} = \frac{\text{bits erróneos}}{\text{bits totales medidos}} \times 1.000.000$$

Ejemplo — `fw0/main.c`

```
ppm = window_errors * 1000000 / WINDOW_BITS
```

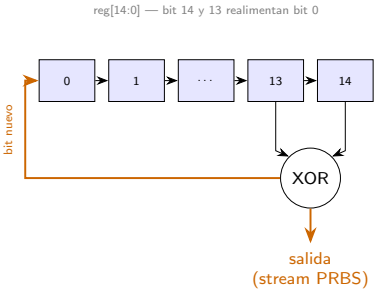
32 errores / 8192 bits $\rightarrow$ ppm $= 32/8192 \times 10^6 \approx$ **3906**

3906 ppm equivale, en promedio, a 1 error cada $\approx$ 256 bits.

PRBS-15: un LFSR con realimentación XOR

LFSR (Linear Feedback Shift Register):

- Registro de desplazamiento de n bits con un XOR de algunos bits (*taps*) realimentado a la entrada
- PRBS-15 (ITU-T O.150) usa $n = 15$: polinomio $x^{15} + x^{14} + 1 \rightarrow$ taps en los bits 14 y 13
- Se repite recién a los $2^{15} - 1 = 32767$ bits. "Parece" aleatorio en cualquier ventana corta
- El bit de salida ES el bit de realimentación: no hace falta lógica extra para generarlo



Una sola implementación, dos roles

`lab/common/prbs.h`: el **generador** avanza con su propia realimentación; el **checker** predice con la misma fórmula pero desplaza el bit **recibido**, no el propio.

Autosincronización del checker

El checker **no conoce la semilla** del generador: carga su registro con los bits **recibidos**. Tras ∼15 bits correctos ya predice igual.

¿Por qué más de un procesador?

Un solo core hace todo:

- Recibe comandos por UART (I/O lento)
- Computa resultados (CPU intensivo)
- Controla LEDs y periféricos
- Todo en un loop secuencial — tareas se bloquean entre sí

Con 2 cores — división de roles:

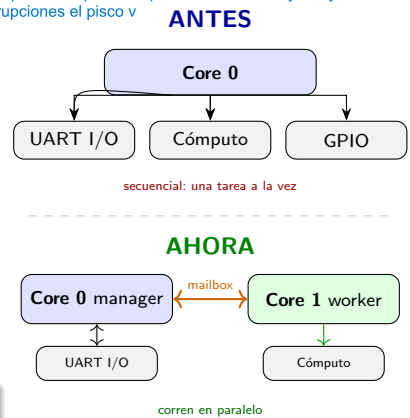
- **Core 0 (manager):** UART + control de flujo
- **Core 1 (worker):** cómputo intensivo
- Ambos corren en paralelo, en hardware

El desafío

¿Cómo le pasa Core 0 un operando a Core 1
y recibe el resultado de vuelta?

mailbox es para que se puedan
enviar mensajes entre ellos por poling

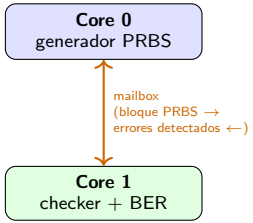
y sino por interrupciones (esto tiene un manejo muy feo de
interrupciones el pisco v



Caso de aplicación: un BERT de 2 cores

Construimos el mismo principio que un BERT (Bit Error Rate Tester):

- **Core 0 (generador):** produce un stream PRBS-15 e inyecta errores a una tasa configurable.
- **Core 1 (checker):** regenera la misma secuencia de forma **autosincronizada** (sin conocer la semilla de Core 0), cuenta los bits que no coinciden y reporta el BER.
- El mailbox transporta el stream por **bloques**, no un único valor: mismo protocolo IDLE/WORK/DONE, más datos por vuelta.



De repente el core 0 va a genera run , un lote de muestras, lo escribe en el mailbox y cambia el estado a work, ahi el core 1 va a estar esperando hasta que el core 0 le dice che pase de IDLE a work

El mailbox: memoria compartida con protocolo

¿Qué es un mailbox?

- Una región de BRAM accesible desde ambos cores
- Acceso por direcciones mapeadas en memoria (0x500000)
- Contiene palabras de datos + una palabra de estado
- El **protocolo software** garantiza acceso exclusivo

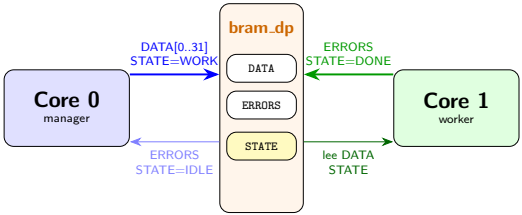
Dirección	Nombre	Quién escribe
0x500000	`MBX_STATE`	ambos
0x500004	`MBX_DATA[0..31]`	Core 0 (bloque PRBS)
0x500084	`MBX_ERRORS`	Core 1

Sin árbitro hardware

No hay semáforo ni mutex en hardware.
El protocolo de 3 fases garantiza que ningún core accede fuera de su turno.

dualport estan mapeados en la misma direccion para ambos procesadores, cuando nosotros construyamos nuestro driver la BUS ADDRES para ambos procesadores, debe ser la misma,

(REGION COMPARTIDA)



State debe ser accesible por ambos procesadoes core 0 lo debería poner en work y el core 1 en IDLE

Protocolo de 3 fases: IDLE → WORK → DONE

Estado	Nombre	Significado
0	`MBX_IDLE`	Mailbox libre; Core 0 puede enviar trabajo
1	`MBX_WORK`	Core 0 cargó datos; Core 1 debe procesar
2	`MBX_DONE`	Core 1 escribió resultado; Core 0 puede leer

Inicio

IDLE

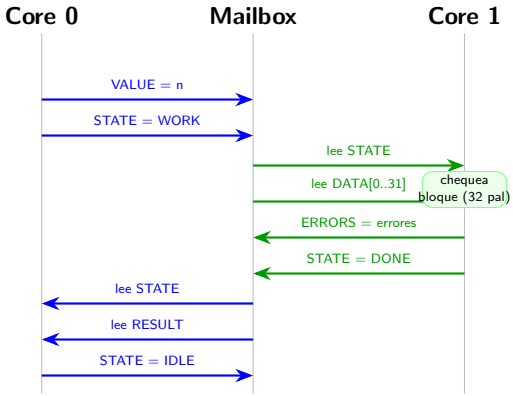
WORK

DONE

Regla fundamental

Cada core escribe en **sus propias palabras**.
Core 0 nunca escribe RESULT;
Core 1 nunca escribe VALUE.
Sólo STATE es compartida entre ambos.

MUTEX, SEMAFOROS
IRQS vs polling

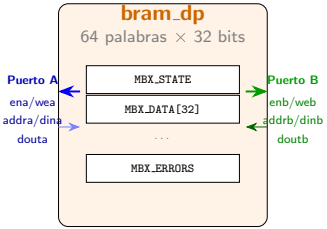


IPC --- Inter processor comunication

Fijar como hacer para guardar espacio en la memoria ram, para guardar datos de la memoria ram y poder comunicar procesos entre ellos

BRAM dual-port: bram_dp.v


```
1  module bram_dp #(
2      parameter DATA_W = 32,
3      parameter ADDR_W = 6        // 64 palabras = 256 bytes
4  )(
5      // Puerto A --- Core 0
6      input               clka, ena, wea,
7      input  [ADDR_W-1:0] addra,
8      input  [DATA_W-1:0] dina,
9      output reg [DATA_W-1:0] douta,
10
11     // Puerto B --- Core 1
12     input               clkb, enb, web,
13     input  [ADDR_W-1:0] addrb,
14     input  [DATA_W-1:0] dinb,
15     output reg [DATA_W-1:0] doutb
16 );
17     reg [DATA_W-1:0] mem [(1<<ADDR_W)-1:0];
18
19     // Puerto A: READ_FIRST (dout = valor anterior)
20     always @(posedge clka)
21         if (ena) begin
22             if (wea) mem[addra] <= dina;
23             douta <= mem[addra];    // lee ANTES de la escritura
24         end
25
26     // Puerto B: idéntico
27     always @(posedge clkb)
28         if (enb) begin
29             if (web) mem[addrb] <= dinb;
30             doutb <= mem[addrb];
31         end
32 endmodule
```

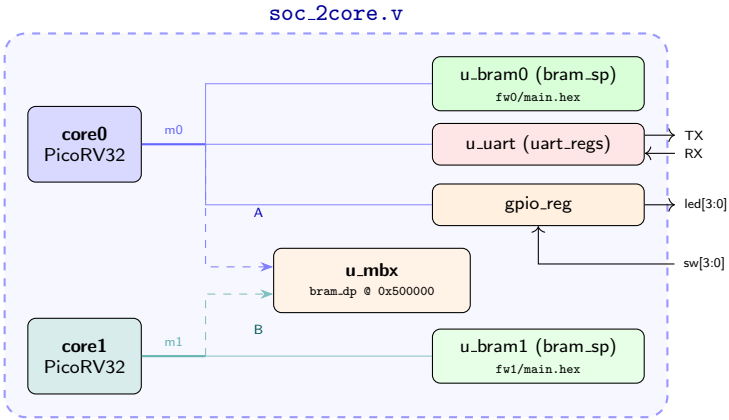


READ_FIRST

dout captura el valor **antes** de la escritura.
Útil para leer-modificar-escribir sin hazards.

Se podria implementar que los procesadores tengan distintos dominios de reloj, uno a 100mhz y otro a la mitad

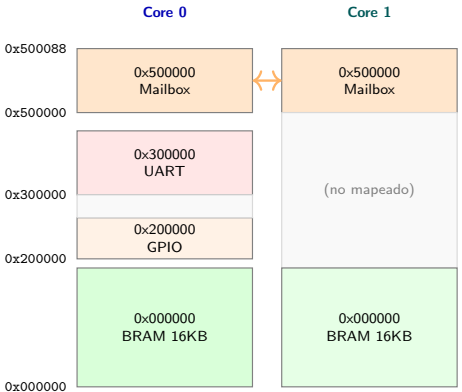
Estructura RTL: soc_2core.v



Mapa de memoria: perspectiva de cada núcleo

Dirección	Periférico	Acceso	Latencia
0x0000–0x3FFF	BRAM privada 16 KB	R/W	1 ciclo
0x200000	GPIO_LEDS (LEDs [3:0])	W	inmediata
0x200004	GPIO_SW (switches [3:0])	R	inmediata
0x300000	UART DATA	R/W	inmediata
0x300004	UART STATUS	R	inmediata
0x500000	MBX_STATE	R/W	1 ciclo
0x500004–0x500083	MBX_DATA[32]	R/W	1 ciclo
0x500084	MBX_ERRORS	R/W	1 ciclo

- GPIO y UART: sólo visibles desde Core 0
- Mailbox: accesible por **ambos cores** vía bram_dp
- Cada core tiene su propia BRAM privada (mismo rango, distinto hardware)



¿Qué es una "ventana" de medición?

Definición — mailbox_prbs.h

```
#define BLOCK_WORDS     32u    // 1024 bits/bloque

/* bloques que arma una ventana de BER */
#define WINDOW_BLOCKS   8u
#define WINDOW_BITS     (WINDOW_BLOCKS * BLOCK_WORDS * 32u)
```

En números

1 bloque = 32 palabras × 32 bits = 1024 bits $\rightarrow$ 1 ventana = WINDOW_BLOCKS bloques = 8 × 1024 = **8192 bits**

¿Por qué agrupar en ventanas?

Core 0 no calcula el BER bloque a bloque: acumula `window_errors` a lo largo de WINDOW_BLOCKS bloques y recién al cerrar la ventana calcula e imprime el BER en ppm. Con pocos bits medidos el BER instantáneo es ruidoso; promediar sobre 8192 bits da una muestra más representativa.

Firmware Core 0: manager — `fw0/main.c`

```c
#include "mailbox_prbs.h"
#include "prbs.h"

#define ERR_PERIOD_BITS 1021u  // primo; 0 = sin errores

void main(void) {
    uart_puts("=== PRBS/BER por mailbox (BERT 2 cores) ===\r\n");

    prbs_gen_t gen;
    prbs_gen_init(&gen, 0x1ACEu);
    uint32_t bit_count = 0, window_blocks = 0,
             window_errors = 0, block_num = 0;

    while (1) {
        while (MBX_STATE != MBX_IDLE);

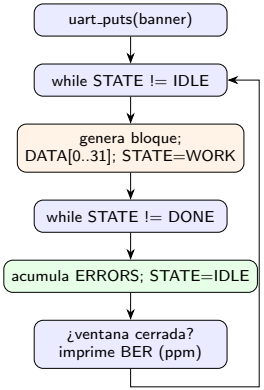
        // Generar un bloque (32 palabras = 1024 bits)
        for (uint32_t i = 0; i < BLOCK_WORDS; i++)
            MBX_DATA[i] = gen_word_with_errors(&gen, &bit_count);

        GPIO = block_num++ & 0xF;
        MBX_STATE = MBX_WORK;

        while (MBX_STATE != MBX_DONE);
        window_errors += MBX_ERRORS;
        MBX_STATE = MBX_IDLE;

        if (++window_blocks >= WINDOW_BLOCKS) {
            uint32_t ppm = (uint32_t)((uint64_t)window_errors
                           * 1000000ULL) / WINDOW_BITS);
            uart_puts("[C0] ventana: BER ~= ");
            uart_print_uint(ppm);
            uart_puts(" ppm\r\n");
            window_blocks = window_errors = 0;
        }
    }
}
```

Flujo de ejecución Core 0:



Firmware Core 1: checker autosincronizante — `fw1/worker.c`

```c
#include "mailbox_prbs.h"
#include "prbs.h"

void main(void) {
  prbs_chk_t chk;
  prbs_chk_init(&chk, 0x0001u);

  while (1) {
    while (MBX_STATE != MBX_WORK);
    uint32_t block_errors = 0;
    for (uint32_t i = 0; i < BLOCK_WORDS; i++)
      block_errors += prbs_chk_word(&chk,
          MBX_DATA[i]);

        MBX_ERRORS = block_errors;
        MBX_STATE  = MBX_DONE;
    }
}
```

aca estamos llamando al driver que se va a encargar de llamar al computo de la PRBS , en esa funcion ()

prbs_with_errros

Características del checker:

- **Sin UART, sin GPIO** — solo mailbox
- Loop infinito: espera → chequea bloque → responde
- **No conoce la semilla de Core 0**: se autosincroniza con los bits recibidos (ver `prbs.h`)
- Primero escribe el resultado (`ERRORS`), luego cambia el estado (`DONE`) ⇒ Core 0 siempre lee un resultado válido

Orden de escritura importa

Correcto: ERRORS → STATE=DONE
Incorrecto: STATE=DONE → ERRORS

BER medido en simulación (ventanas reales)

Ventana 1: 32 errores / 8192 bits → 3906 ppm (transitorio de sync)
Ventana 2: 24 errores / 8192 bits → 2929 ppm
Ventana 3: 24 errores / 8192 bits → 2929 ppm (BER en régimen)
La primera ventana mide más alto: el checker todavía no terminó de autosincronizarse en los primeros bits. Valores para SW[3:0]=1000 (nivel 8), el que fija el testbench.

SW[3:0]: 16 niveles de inyección de error

instrucciones atomicas googlear

GPIO_SW — periférico MMIO RO

- Mismo periférico y mismo driver `sw.h` que se implementó. Lo lee el Core 0 en vez del único core que había
- Mapeado en 0x200004 (offset dentro de la misma región GPIO que los LEDs, 0x200000)
- `soc_2core.v` decodifica `addr[2]`: offset 0x0 = LEDs (W), offset 0x4 = SW (R)
- Los 4 switches físicos de la Arty A7 seleccionan uno de **16 niveles** de tasa de error
- Cambiar los switches en la placa cambia la tasa **sin recompilar ni resintetizar**: el firmware relee SW[3:0] al inicio de cada bloque

Driver: `fw0/sw.h`

```
#define GPIO_SW \
  (*(volatile uint32_t *)0x00200004)

static inline uint32_t sw_read(void) {
    return GPIO_SW & 0xFu; }

static const uint32_t ERR_PERIOD_TABLE[16] = {
    0, 100003, 50021, 25013, 16001,
    8009, 4001, 2003, 1021, 509,
    251, 127, 61, 31, 17, 7
};
static inline uint32_t sw_err_period(void) {
    return ERR_PERIOD_TABLE[sw_read()]; }
```

SW[3:0]	periodo (bits)	BER aprox.
0000	sin errores	0 ppm
0001	100003	∼10 ppm
0100	16001	∼62 ppm
1000	1021	∼980 ppm
1100	61	∼16400 ppm
1111	7	∼142900 ppm

Por qué en `main.c`, no en `fw1/`

Sólo Core 0 tiene GPIO en su mapa de memoria: es quien genera el stream PRBS y decide cuándo invertir un bit, así que es el único lugar donde tiene sentido leer los switches.

Cada ventana cerrada imprime el nivel de SW usado (`sw_read()`) junto al BER medido, para confirmar en vivo que el cambio de switches tuvo efecto.

Orden de escrituras: ¿por qué importa?

Protocolo Core 1 — versión incorrecta:

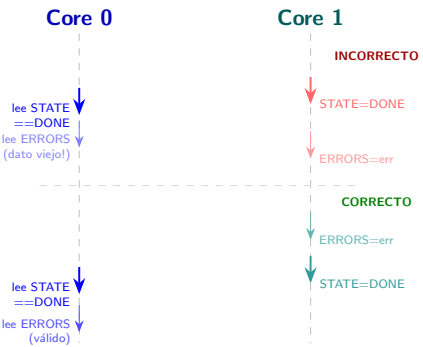
```
1 // MAL: señaliza ANTES de escribir el resultado
2 MBX_STATE  = MBX_DONE;   // Core 0 ya ve DONE
3 MBX_ERRORS = block_errors;          // ...pero ERRORS
      aún no
```

Race condition

Core 0 detecta `STATE==DONE` y lee `ERRORS` *antes* de que Core 1 lo haya escrito.
Lee el valor anterior — dato incorrecto.

Protocolo Core 1 — versión correcta:

```
1 // BIEN: dato antes que la señal
2 MBX_ERRORS = block_errors;          // 1. escribir
      dato
3 MBX_STATE  = MBX_DONE;   // 2. señalizar
```



static
volatile
extern

volatile en C: clave del protocolo

Static solo es accesible por sus mismos archivos de compilacion, es decir archivos que esten en su misma rama digamos.

inline: evita que se descomponga en una funcion separada, ya que es corta (es para evitar ciclos de overhead para saltar a la funcion)

volatile previene reordenamiento del compilador:

```
// mailbox_prbs.h: cada campo es volatile uint32_t *
#define MBX_STATE
(*(volatile uint32_t *)(MBX_BASE + 0x00))
#define MBX_DATA
((volatile uint32_t *)(MBX_BASE + 0x04))
// 32 palabras
#define MBX_ERRORS
 (*(volatile uint32_t *)(MBX_BASE + 0x84))

// Sin volatile, gcc -O2 puede reordenar:
MBX_ERRORS = block_errors;       // store 1
MBX_STATE  = MBX_DONE; // store 2
// --> con volatile, el compilador
//     respeta este orden

// Sin volatile, incluso el polling
// puede eliminarse por el compilador:
while (MBX_STATE != MBX_DONE); // siempre necesita
    releer
```

Regla para este lab

- Todo acceso al mailbox debe ser `volatile` (ya está en `mailbox_prbs.h`)
- Escribir el dato **antes** del estado
- Leer el estado **antes** de consumir el dato

PicoRV32: sin ejecución fuera de orden

- PicoRV32 es en orden estricto — el hardware no reordena accesos a memoria
- El riesgo es **solo del compilador**
- `volatile` es suficiente sin `fence` (aunque `fence` sería correcto igual)

Testbench: sim/tb_soc_2core.v

```
`timescale 1ns/1ps
module tb_soc_2core;
  localparam CLK_HZ   = 100_000_000;
  localparam SIM_BAUD = 1_000_000;  // 100 ciclos/bit
  localparam BIT_CYC  = CLK_HZ / SIM_BAUD;

  reg clk, resetn;
  wire [3:0] led;
  wire tx, trap;

  soc_2core #(.CLK_HZ(CLK_HZ), .BAUD(SIM_BAUD)) dut (
    .clk(clk), .resetn(resetn), .led(led),
    .rx(1'b1), .tx(tx), .trap(trap)
  );

  // Decodificador 8N1: captura 1 byte del pin TX
  task uart_capture_byte;
    output [7:0] data_out; integer i;
    begin
      @(negedge tx);                    // start bit
      repeat(BIT_CYC/2) @(posedge clk);  // centro
      for (i=0; i<8; i=i+1) begin
        repeat(BIT_CYC) @(posedge clk);
        data_out[i] = tx;                // LSB primero
      end
      repeat(BIT_CYC) @(posedge clk);    // stop bit
    end
  endtask

  reg [7:0] rx_byte; integer char_count;
  initial begin
    char_count = 0; @(posedge resetn);
    forever begin
      uart_capture_byte(rx_byte);
      $write("%c", rx_byte);
      if (++char_count >= 600) $finish;
    end
  end
endmodule
```

Salida esperada en simulación:

```
=== Jornada 4: PRBS/BER por mailbox (BERT de 2 cores)
    ===
Core 0: generador PRBS + manager UART  |  Core 1:
    checker
Bloque = 1024 bits  |  Ventana = 8192 bits

[C0] ventana cerrada: 32 errores / 8192 bits -> BER ~=
     3906 ppm
[C0] ventana cerrada: 24 errores / 8192 bits -> BER ~=
     2929 ppm
[C0] ventana cerrada: 24 errores / 8192 bits -> BER ~=
     2929 ppm
```

Simulación vs. hardware real

En `iverilog` cada ventana tarda minutos reales (1024 bits $\times$ 2 cores, con LFSR bit a bit en software). En la Arty A7 real, a 100 MHz, el mismo cálculo es instantáneo.

Comandos:

```
make sim     # iverilog + vvp
make wave    # GTKWave con VCD
make synth   # bitstream para Arty A7
```

Laboratorio: SoC 2-Core con Mailbox — BERT de PRBS/BER

Archivos entregados (completos):

- `rtl/picorv32.v` — procesador PicoRV32
- `rtl/bram_sp.v` — BRAM single-port con byte enables
- `rtl/bram_dp.v` — BRAM dual-port (mailbox)
- `rtl/uart_*.v`, `uart_regs.v` — stack UART (de Jornada 2)
- `fw0/uart.h` — driver UART (sin cambios)
- `fw0/sw.h` — driver de los switches SW[3:0] (GPIO_SW)
- `fw0/prbs.h`, `fw1/prbs.h` — LFSR generador/checker (idéntico en ambos)
- `sim/tb_soc_2core.v` — testbench con captura UART (SW fijo en nivel 8)

Partes a completar

1. `rtl/soc_2core.v` (7 TODOs: A–G):
   - **A** — Decodificador: `m0_sel_mbx` y `m1_sel_mbx`
   - **B** — Instanciar `bram_dp` (puertos A y B)
   - **C/D** — `mbx0_ready_r` para Core 0: registrar y conectar
   - **E** — ídem `mbx1_ready_r` para Core 1
   - **F/G** — `mbx_rdata` en mux de Core 0 y Core 1
2. `fw0/main.c`: generar el bloque PRBS e inyectar errores (`gen_word_with_errors`), leyendo la tasa en vivo de `sw_err_period()` (`sw.h`), cerrar la ventana y reportar BER por UART.
3. `fw1/worker.c`: chequear cada bloque recibido con `prbs_chk_word` y devolver `MBX_ERRORS`.

Flujo de trabajo:

```
# 1. Compilar firmwares
make hex
# 2. Simular (iverilog) -- tarda varios
#    minutos reales por ventana
make sim
# Esperado:
# [C0] ventana cerrada: N errores /
#      8192 bits -> BER ~= X ppm
# 3. Ver formas de onda
make wave
# 4. Sintetizar para Arty A7
make synth
# 5. Programar la placa
make program
```

Verificación mínima

Con `ERR_PERIOD_BITS=0` (sin inyección), la **primera** ventana puede mostrar algunos errores (transitorio de autosincronización del checker); a partir de la segunda ventana el BER debe caer a **0 ppm**: confirma que no hay bugs de protocolo.